

Distributed Implementation of Full Configuration Interaction for One Trillion Determinants

Hong Gao,* Satoshi Imamura, Akihiko Kasagi, and Eiji Yoshida

Cite This: *J. Chem. Theory Comput.* 2024, 20, 1185–1192

Read Online

ACCESS |



Metrics & More

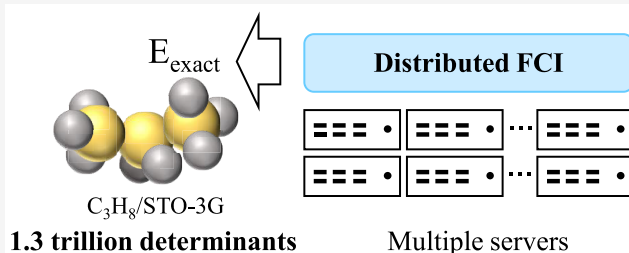


Article Recommendations



Supporting Information

ABSTRACT: Full configuration interaction (FCI) can provide an exact molecular ground-state energy within a given basis set and serve as a benchmark for approximate methods in quantum chemical calculations, including the emerging variational quantum eigensolver. However, its exponential computational and memory requirements easily exceed the capability of a single server and limit its applicability to large molecules. In this paper, we present a distributed FCI implementation employing a hybrid parallelization scheme with multithreading and multiprocessing to expand FCI's applicability. We optimize this scheme to minimize the bottlenecks arising from interprocess communications and interthread data management. Our implementation achieves higher scalability than the naive combination of prior works and successfully calculates the exact energy of C_3H_8 /STO-3G with 1.31 trillion determinants, which is the largest FCI calculation to the best of our knowledge. Furthermore, we provide a comprehensive list of FCI results with 136 combinations of molecules and basis sets for future evaluation and development of approximate methods.



1. INTRODUCTION

Ground-state energy calculation is a basic task in quantum chemistry because it provides insights into molecular characteristics, such as the stability, chemical reactions, electronic structure, and total energy of a quantum system. It has been widely applied in materials science, condensed matter physics, biochemistry, drug discovery, and so forth.^{1–4}

Various methods exist for calculating ground-state energy: Hartree–Fock, density functional theory, Møller–Plesset perturbation theory, coupled cluster, and so forth.^{5–8} In particular, the full configuration interaction (FCI) method serves as an accuracy benchmark to evaluate approximate methods because it theoretically provides the exact solution for the Schrödinger equation within a given basis set by accounting for all possible electronic configurations. However, its applicability is limited to small molecules and basis sets due to the exponential computational and memory complexity according to the number of electrons and molecular orbitals (MOs).^{9,10}

Since the 1980s, a lot of effort has been made to expand the FCI's applicability. Handy et al. decomposed the two-electron part of the Hamiltonian as a sum over intermediate singly excited states.^{11–14} Olsen et al. improved this theory and successfully calculated over one billion Slater determinants, as shown by the left red point in Figure 1.¹⁵

In the early 2000s, the development of supercomputers encouraged distributed FCI implementations using message passing interface (MPI) and increased the number of determinants to ten billion, as shown by the middle and right red points in Figure 1.^{16–18} However, distributed implementa-

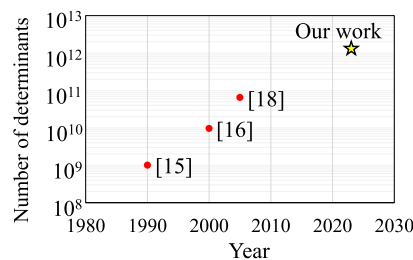


Figure 1. Expansion of FCI applicability.

tions suffer from longer interprocess communication times, as the number of processes increases. For instance, Ansaloni et al. observed that with 128 processes, the communication time reached 26,000 s, while the calculation time was 500 s.¹⁶ Unfortunately, energy results that can be used to evaluate the accuracy of appropriate methods were not provided in these works.

Since 2005, the FCI research has shifted toward the development of two approximate CI methods: selected CI and FCI quantum Monte Carlo (FCIQMC). The selected CI methods select only important portions of Slater determi-

Received: October 27, 2023

Revised: December 24, 2023

Accepted: January 12, 2024

Published: February 5, 2024



Table 1. Comprehensive Comparison between Prior Works and Our Work

	SC '05 ¹⁸	PySCF '20 ⁴⁰	our work
molecule/basis set	C2/cc-pVTZ(+1s,1p)	-	C3H8/STO-3G
active space ^a	(8 elecs, 66 orbs)	(18 elecs, 18 orbs)	(26 elecs, 23 orbs)
full space	NO	YES	YES
# of determinants	6.5×10^{10}	2.4×10^9	1.3×10^{12}
# of servers	108	1	256
energy results	not provided	not provided	136 results provided

^aActive space is defined with the number of electrons and the number of orbitals.

nants,^{19–32} while the FCIQMC methods sample the FCI wave function using a stochastic approach.^{31–38} Although these methods can be applied to larger molecules compared with the exact FCI, their accuracy depends on the number of selected configurations.

In 2017, a multithreaded FCI implementation with open multi-processing (OpenMP) was presented in an open-source Python-based Simulations of Chemistry Framework (PySCF) to fully utilize the potential of multicore processors.³⁹ However, since there is no distributed FCI implementation in PySCF, its applicability is limited to 18 electrons with 18 orbitals, which corresponds to one billion Slater determinants.⁴⁰ Although there is a MPI implementation of PySCF called mpi4pyscf,⁴¹ FCI is not included in it.

More recently, the development of emerging quantum chemical calculations (QCC) methods has further amplified the significance of large-scale exact FCI. The advent of noisy intermediate-scale quantum computers has driven the development of quantum computing-based methods such as the variational quantum eigensolver (VQE) and quantum Monte Carlo (QMC).^{42–52} These methods have demonstrated superior accuracy compared to traditional QCC methods, such as coupled-cluster singles and doubles with perturbative triples [CCSD(T)] in some cases. However, the exact FCI energies necessary for their accuracy evaluation are lacking, even for relatively large molecules such as C₃H₆/STO-3G.⁵³

In this paper, to calculate the exact ground-state energies of larger molecules than prior works, we present a distributed FCI implementation using a hybrid MPI-OpenMP scheme based on the OpenMP implementation in PySCF. We find that a naive hybrid MPI-OpenMP implementation based on prior works^{17,39} suffers from significant MPI communication time and processing time required for handling intermediate buffers when the number of processes is large. To address this issue, we optimize MPI communication and propose thread-safe cyclic data management to eliminate the intermediate buffers.

Compared to the naive distributed implementation, our optimized implementation achieves a 55% time reduction for BH₃/STO-3G. We also confirm that our implementation scales better to the number of processes than the naive implementation. As a result, we successfully calculated the exact ground-state energy of C₃H₈/STO-3G, comprising 1.3 trillion (10^{12}) determinants, by running 512 processes on 256 servers in 113.6 h. To the best of our knowledge, this is the largest-scale FCI calculation, as shown by the star point in Figure 1. Table 1 also shows the comprehensive comparison between our work and prior works. Unlike prior works that limited the active space, we calculate the exact ground-state energies of 136 combinations of molecules and basis sets in the full active space and present all of the results in the Supporting Information.

We believe that this work would contribute to the accuracy evaluation of approximate QCC methods for large molecules

that have not ever been evaluated exactly. For instance, Table 2 shows an example to evaluate the accuracy of CCSD(T), VQE,

Table 2. Example to Evaluate the Accuracy of Approximate Methods for C₃H₆/STO-3g Using Our Distributed FCI Implementation

method	energy [Hartree]	error from FCI [Hartree]
distributed FCI	−115.887177	-
CCSD(T)	−115.886414	0.000763
VQE ^a	−113.832597	2.054580
QMC ⁵³	−115.886571	0.000606

^aExecuted with a noise-less AerSimulator in Qiskit⁵⁴ in an active space of 8 electronics and 8 orbitals.

and QMC⁵³ for C₃H₆/STO-3G. While it was previously impossible to determine the most accurate method without the exact FCI energy, our distributed implementation now enables this by executing FCI on 24 servers in 6 h (see Section 3 for the experimental setup). We now know that the QMC method achieves an accuracy higher than that of CCSD(T) and VQE.

The organization of this article is as follows. Section 2 provides background knowledge on FCI and details of our hybrid MPI-OpenMP implementation. Section 3 presents the evaluation results of our implementation. Finally, Section 4 concludes this work. A comprehensive list of energy results can be found in the Supporting Information.

2. METHODS

2.1. FCI Theory. The goal of FCI is to find an exact solution to the electronic Schrödinger equation

$$\hat{H}|\Psi\rangle = E|\Psi\rangle \quad (1)$$

where $\hat{H}$ represents the Hamiltonian operator and Ψ stands for a wave function. Knowles and Handy proposed an algorithm that represents Ψ with a linear combination of Slater determinants Φ_i .¹⁴ In their algorithm, Slater determinants are constructed by combining α -strings and β -strings, which represent the arrangement of α -spin (spin-up) and β -spin (spin-down) electrons in MOs, respectively. The α - and β -strings are bit-strings, where “1” indicates an occupied MO and “0” signifies an empty MO. The total number of Slater determinants N_{det} is calculated as follows (assuming a closed-shell singlet state)

$$N_{\text{det}} = \binom{N_o}{N_e}^2 = \left(\frac{N_o!}{(N_o - N_e)!N_e!} \right)^2 \quad (2)$$

where N_o represents the number of MOs and $N_e = N_\alpha = N_\beta$ represents the number of electrons with α -spin, which is half of the total number of electrons.

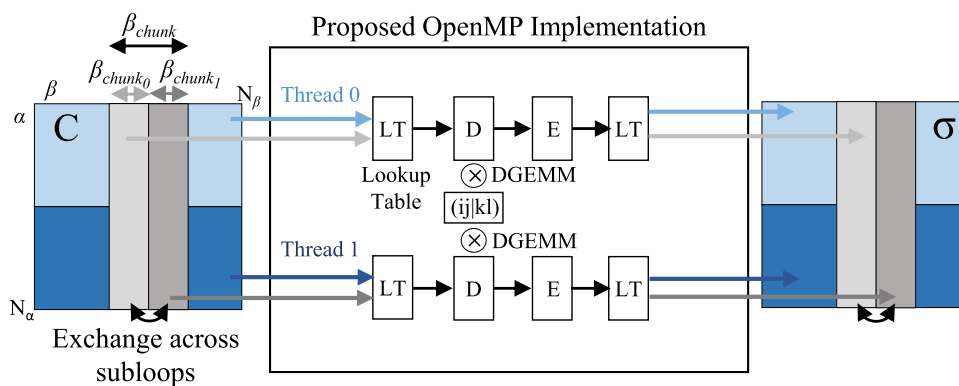


Figure 2. Our proposed OpenMP implementation of FCI. This is an example with two threads.

The exact solution of eq 1 can be calculated by diagonalizing a Hamiltonian matrix H , where each element is calculated as $H_{ij} = \langle \Phi_i | \hat{H} | \Phi_j \rangle$. However, the direct diagonalization of the entire H is considered impractical because the size of H is the same as N_{det} , which increases exponentially with the number of electrons and MOs. Thus, variational principles such as the Davidson diagonalization method are typically employed for FCI.⁵⁵ They operate on subspaces of H while progressively refining the approximation of the eigenvalues and eigenvectors. Specifically, H is projected onto a set of subspace vectors, and a subspace matrix is iteratively expanded by appending components calculated with subspace vectors. When the number of subspace vectors exceeds a predetermined threshold, the subspace matrix is reset to limit the memory requirement.

The most time-consuming step in each Davidson iteration is the calculation of a subspace vector σ with a Hamiltonian matrix element H_{ij} and CI coefficient vector c_j as follows

$$\sigma_i = \sum_j H_{ij} c_j \quad (3)$$

The algorithm shows the pseudocode of this calculation.^{12,56} It includes five steps: forming the D matrix with α -contributions (lines 1–8) and β -contributions (lines 9–16), calculating the E matrix through matrix–matrix multiplication (line 17), and constructing σ with α -contributions (lines 18–25) and β -contributions (lines 26–33). α' - and β' -strings represent α - and β -strings that have at most one occupation different from that of α - and β -strings, respectively. Given an α -string, the corresponding α' -strings, indexes ij (i.e., $\text{link}[\alpha, \alpha']$), and one-electronic coupling coefficients $\gamma[\alpha, \alpha']$ are stored locally. For instance, such information for both α - and β -strings is preserved in a lookup table (LT) in the FCI implementation of PySCF. Therefore, the α - and β -contributions to D or σ can be calculated by referring to the LT, multiplying γ with the specified element of C or E , and subsequently adding the result to the specified element of D or σ . Matrix–matrix multiplication to calculate E can be executed with well-known libraries, such as the BLAS DGEMM library.

Algorithm 1 The pseudo code of Knowles and Handy's algorithm^{12,56}

```

1: for  $\alpha$ -strings do
2:   for  $\alpha'$ -strings do
3:     for  $\beta$ -strings do
4:        $ij \leftarrow \text{link}[\alpha, \alpha']$ 
5:        $D[ij, \alpha, \beta] += \gamma[\alpha, \alpha'] C[\alpha', \beta]$ 
6:     end for
7:   end for
8: end for
9: for  $\beta$ -strings do
10:  for  $\beta'$ -strings do
11:    for  $\alpha$ -strings do
12:       $ij \leftarrow \text{link}[\beta, \beta']$ 
13:       $D[ij, \alpha, \beta] += \gamma[\beta, \beta'] C[\alpha, \beta']$ 
14:    end for
15:  end for
16: end for
17: matrix multiply :
     $E[kl, \alpha, \beta] = (ij|kl) D[ij, \alpha, \beta]$ 
18: for  $\alpha$ -strings do
19:  for  $\alpha'$ -strings do
20:    for  $\beta$ -strings do
21:       $ij \leftarrow \text{link}[\alpha, \alpha']$ 
22:       $\sigma[\alpha', \beta] += \gamma[\alpha, \alpha'] E[ij, \alpha, \beta]$ 
23:    end for
24:  end for
25: end for
26: for  $\beta$ -strings do
27:  for  $\beta'$ -strings do
28:    for  $\alpha$ -strings do
29:       $ij \leftarrow \text{link}[\beta, \beta']$ 
30:       $\sigma[\alpha, \beta'] += \gamma[\beta, \beta'] E[ij, \alpha, \beta]$ 
31:    end for
32:  end for
33: end for

```

2.2. Optimized Hybrid Parallel Implementation. We employ a hybrid parallelization scheme combining multiprocessing with MPI and multithreading with OpenMP to fully utilize the potential of multiple servers equipped with multicore processors. In the following subsections, we first present our optimized OpenMP implementation and then describe our optimized hybrid MPI-OpenMP implementation. In addition, we introduce a restart function for long-time executions.

2.2.1. Optimized OpenMP Implementation. We implement a multithreaded FCI code with OpenMP based on the PySCF implementation, as shown in the algorithm and Figure 2. The columns (β -strings) of C and σ are partitioned into fixed-size β chunks to limit the amount of interprocess communication at a

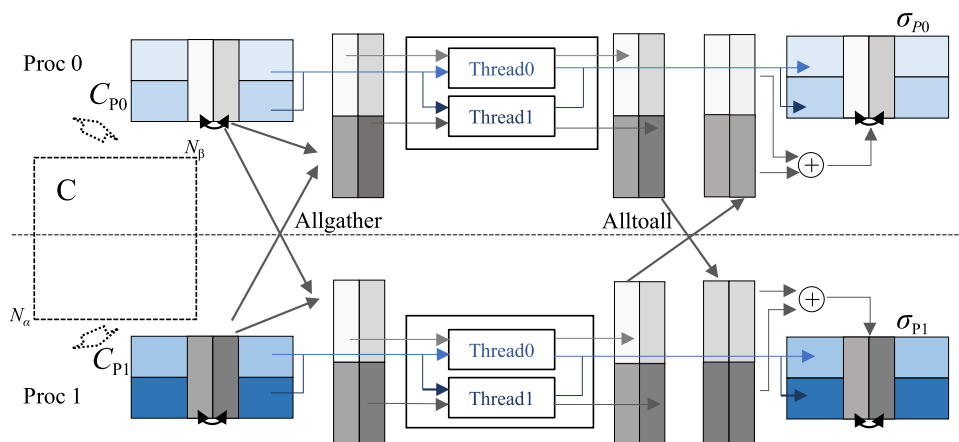


Figure 3. Our optimized hybrid MPI-OpenMP implementation. This is an example with two processes, each of which has two threads.

time (see Section 2.2.2 in detail). For efficient multithreaded processing, we apply a thread-safe cyclic data assignment method that further divides a β chunk into N_{threads} blocks and calculates the β -contributions in each block ($\beta\text{chunk}_{\text{bid}}$) in parallel with multiple threads. The blocks processed by multiple threads are exchanged across subloops to enable different threads to safely handle different blocks in turn. This optimization is a major improvement from the PySCF implementation, where multiple threads concurrently process the same β chunk using thread-local intermediate buffers, and the elements of σ are calculated by adding these buffers. The PySCF implementation suffers from a significant time overhead to reset and add the intermediate buffers, whereas our thread-safe cyclic data assignment eliminates them and enables each thread to directly store the elements of σ .

Algorithm 2 The pseudo code of our optimized OpenMP implementation

```

1: for  $\beta$ chunks do
2:   # pragma omp parallel
3:   {
4:     tid = omp_get_thread_num()
5:     Nthreads = omp_get_num_threads()
6:     for subloop in Nthreads do
7:       bid = (subloop + tid)%Nthreads
8:       # pragma omp for
9:       for  $\alpha$ -strings do
10:        calculate  $\alpha$ -parts to D
11:      end for
12:      calculate  $\beta$ -parts to D in  $\beta\text{chunk}_{\text{bid}}$ 
13:      matrix multiply:  $E = (ij|kl)D$ 
14:      # pragma omp for
15:      for  $\alpha$ -strings do
16:        calculate  $\alpha$ -parts to  $\sigma$ 
17:      end for
18:      calculate  $\beta$ -parts to  $\sigma$  in  $\beta\text{chunk}_{\text{bid}}$ 
19:    end for
20:  }
21: end for

```

On the other hand, the rows (α -strings) of C and σ are distributed to multiple threads and processed in parallel. This is the same scheme as the PySCF implementation. Since there is no data dependency between different α -strings, this scheme can also be applied to our MPI parallelization (see Section 2.2.2 in detail).

2.2.2. Optimized Hybrid MPI-OpenMP Implementation. For molecules whose computational and memory requirements

exceed the capability of a single server, we distribute C and σ to multiple processes and run them on multiple servers. In such a distributed implementation, interprocess communication with MPI is a typical performance bottleneck. In fact, prior distributed implementations suffered from a significant communication time as the number of servers increased.^{16,17}

Figure 3 illustrates the overview of our hybrid MPI-OpenMP implementation. We apply a static load balancing scheme presented by Gan and Harrison, as shown in Figure 4a.¹⁷ It

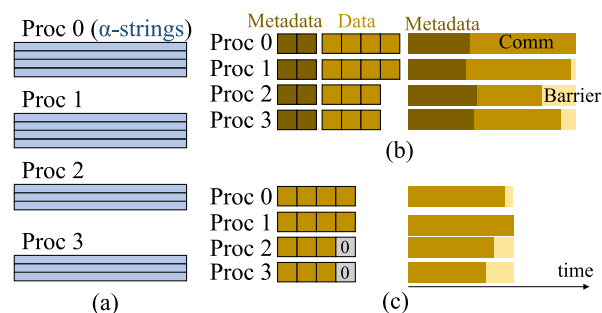


Figure 4. Examples of (a) static load balancing, (b) Allgather communication, and (c) Allgather communication.

evenly distributes C into multiple processes by row (e.g., C_{P0} and C_{P1} for two processes in this figure). If there are remaining rows, they are further distributed evenly among processes so that the load difference between processes is at most one row. The elements of a β chunk are gathered from all processes, and then each process computes each fragment locally with multiple threads, as mentioned in Section 2.2.1. Finally, the elements of σ are calculated by aggregating the results from all processes.

To gather the elements of a β chunk from all processes, we use the Allgather function in MPI. Since a prior MPI implementation¹⁷ used the Allgather function, we show the difference between them in Figure 4. Allgather can communicate variable-size data by including metadata containing the data type and length, which simplifies MPI programming. However, it must communicate the metadata in addition to the data itself and can cause an imbalance of communication among multiple processes. In an imbalanced case, processes handling less data must wait for those handling more data at a barrier point. In contrast, Allgather requires no metadata by communicating fixed-size data and can avoid a communication imbalance. Note that data must be aligned by padding with zeros in an

unnecessary region. The combination of static load balancing in the FCI code and Allgather can minimize the time overhead of MPI communication.

To aggregate the results from all processes for calculating the elements of σ , we use the Alltoall function in MPI. In contrast, the prior MPI implementation¹⁷ combined the Reduce and Scatter functions for this purpose. Figure 5 compares the

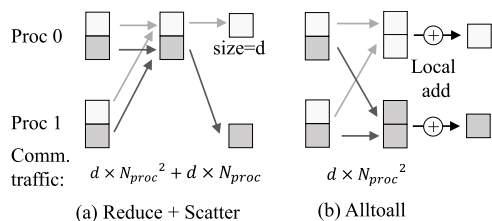


Figure 5. Comparison between Reduce + Scatter and Alltoall.

amounts of communication traffic between them. Supposing each process computes a data block whose size is d , the prior implementation needs to gather data blocks from all processes to a single process and add them with Reduce, followed by Scatter, which distributes the results among all processes. Thus, the total amount of communication traffic is $(d \times N_{procs}^2 + d \times N_{procs})$. In contrast, our MPI implementation reduces it to $d \times N_{procs}^2$ with Alltoall, which only exchanges data blocks each process does not have. Although we need to add all data blocks after Alltoall, it can be performed locally within each process without interprocess communications.

Moreover, we optimize the size of each block in a β chunk (i.e., the size of β chunk_{bid}) that is processed by each thread. Figure 6

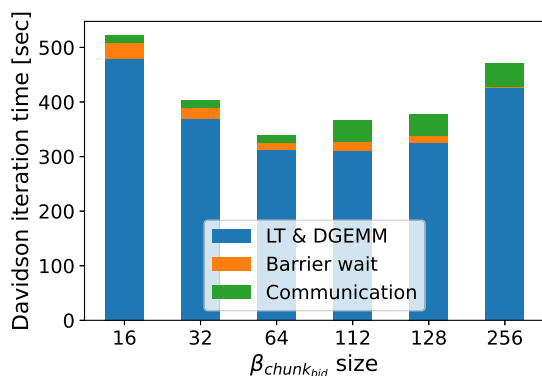


Figure 6. Execution time of one Davidson iteration with the different size of β chunk_{bid} for C₂N₂/STO-3G with four processes running on two nodes.

shows the execution time of one Davidson iteration with different sizes of β chunk_{bid} for C₂N₂/STO-3G with four processes running on two nodes. We observe that the time for accessing LT and performing DGEMM is minimized when the β chunk_{bid} size is 64 or 112. Note that the default size in PySCF is 112. On the other hand, the communication time is minimized when the β chunk_{bid} size is 64. Thus, we set the β chunk_{bid} size to 64 to minimize the time of each Davidson iteration.

2.2.3. Restart Function. For large-scale FCI calculations, FCI can take a few days and abort due to an unexpected system shutdown. Therefore, we implement a restart function based on the out-of-core execution of the PySCF implementation, where the subspaces of C and σ are dumped to files. In our FCI implementation, each process saves the intermediate result

calculated with the dumped subspaces in the latest Davidson iteration into files specified by a target molecule, a basis set, and a Davidson iteration count. When FCI is reexecuted for the same molecule and basis set, it automatically restarts from the latest Davidson iteration by identifying the saved files and loading the intermediate result from them.

3. RESULTS AND DISCUSSION

In this section, we evaluate the execution time and scalability of our optimized distributed FCI implementation by comparing it with a naive combination of prior works. Then, we also validate its exactness by comparing the calculated energies between our implementation and PySCF. Finally, we discuss the time overhead of the restart function of our implementation. In addition, we calculate the ground-state energies of 136 combinations of molecules and basis sets with the molecular geometries obtained from Computational Chemistry Comparison and Benchmark DataBase (CCCBDB).⁵⁷ All of the results are listed in the Supporting Information.

For our evaluation, we use the “V-nodes” of the AI Bridging Cloud Infrastructure (ABCI) supercomputer.⁵⁸ Each node contains two Intel Xeon Gold 6148 CPUs, 384 GB of memory, a 1.6 TB local NVMe SSD, and two InfiniBand EDR cards. In all of the evaluations with multiple processes, we run two processes on each node. Each process operates with 40 threads on each CPU, which has 40 logical cores with Hyper-Threading technology enabled. In addition, the 7.68 TB network-attached NVMe HDD is used to save data for restarts.

3.1. Time Evaluation. We compare the overall execution time of FCI between the naive combination of prior works^{17,39} and our optimized hybrid MPI-OpenMP implementation for BH₃/cc-pVDZ, which has 5.6×10^8 determinants. 64 processes run on 32 servers in this experiment. As shown in Figure 7, our

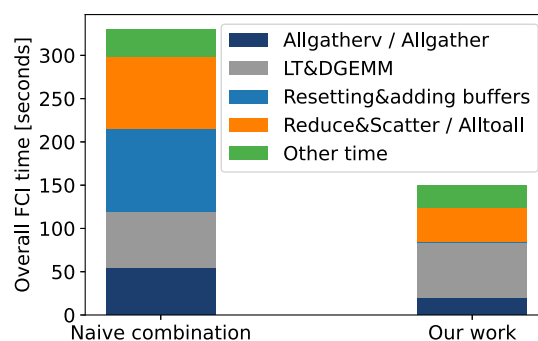


Figure 7. Overall execution time of FCI for BH₃/cc-pVDZ with the naive combination of prior works^{17,39} and our optimized implementation. 64 processes run on 32 servers.

optimized implementation reduces the overall time from 331 to 150 s (by 55%). While the naive combination takes 95.6 s to reset and add the intermediate buffers (light blue), our cyclic thread-safe data assignment method completely eliminates them and takes only 1.6 s to reset buffers for MPI communication. Moreover, our MPI implementation with Allgather and Alltoall significantly reduces the interprocess communication time (dark blue plus orange) from 139 to 56 s compared to the naive combination of Allgather and Reduce and Scatter.

3.2. Scalability Evaluation. Next, we compare the strong scalability for LiH/cc-pVQZ with 1.3×10^7 determinants and NaBH₄/STO-3G with 1.9×10^9 determinants. Figure 8 shows the speedup with multiple processes over a single process of our

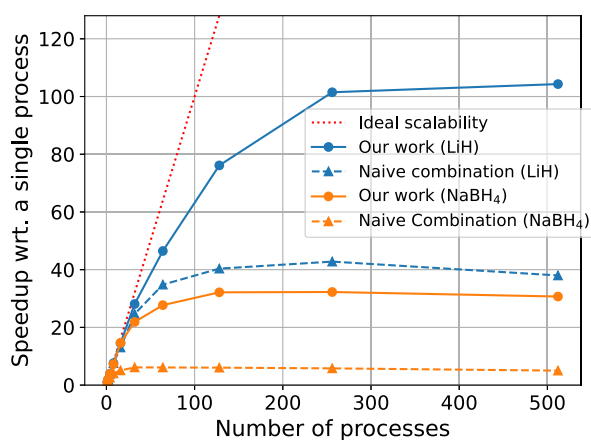


Figure 8. Strong scalability to the number of processes for LiH/cc-pVQZ and NaBH₄/STO-3G. The y-axis indicates the speedup with n processes over a single process, and the red dotted line shows the ideal n times speedup.

optimized implementation and the naive combination of prior works.^{17,39} We can see that our implementation achieves higher scalability than the naive combination. Especially, our implementation scales to 512 processes running on 256 nodes for LiH/cc-pVQZ where the amount of interprocess communication is relatively small. Moreover, it also scales to 128 processes running on 64 nodes for NaBH₄/STO-3G, where the amount of interprocess communication is relatively large, whereas the naive combination fails to scale with more than 32 processes.

3.3. Exactness Validation. To validate the exactness of our hybrid FCI implementation, we compare the ground-state energies of 105 combinations of molecules and basis sets with those of the PySCF implementation. The largest energy difference is only 8.27×10^{-10} , which is negligible for FCI calculation. The detailed results are provided in the [Supporting Information](#).

3.4. Restart Time Evaluation. The time taken to load the intermediate results from specific files for restart depends on the number of processes and the size of C and σ . The largest size evaluated in this work is 19 TB for C₃H₈/STO-3G with 512 processes, where the overall execution time of FCI is 113.6 h. For this experiment, we restart the FCI execution four times, and the total restart time is 3342 s. Thus, the time overhead of the restarts is only 0.8%. To validate the exactness of FCI across restarts, we compare the energies between the cases with four restarts and no restart for C₂H₄O₂/STO-3G having 5.4×10^{11} determinants and confirm that the energy difference is only 2×10^{-11} .

4. CONCLUSIONS

In this work, we present a hybrid MPI-OpenMP implementation of the FCI to expand its applicability to large molecules. We apply a cyclic thread-safe data assignment method to eliminate intermediate buffers for efficient multithreading and optimize interprocess communication using the Allgather and Alltoall functions in MPI. Compared to the naive combination of prior MPI implementations, our optimized hybrid implementation reduces the FCI execution time by 55% with 64 processes and achieves a higher scalability to the number of processes. Consequently, we successfully performed the FCI calculation for C₃H₈/STO-3G with 1.3 trillion determinants, which is the largest scale to our knowledge. Furthermore, we provide the

exact ground-state energies of 136 combinations of molecules and basis sets calculated with our implementation. We hope it will be helpful for future development and evaluation of emerging approximate methods in QCC.

■ ASSOCIATED CONTENT

Supporting Information

The Supporting Information is available free of charge at <https://pubs.acs.org/doi/10.1021/acs.jctc.3c01190>.

Ground-state energies calculated with the PySCF implementation and our hybrid MPI-OpenMP implementation of FCI and geometries of calculated molecules ([PDF](#))

■ AUTHOR INFORMATION

Corresponding Author

Hong Gao – Computing Laboratory, Fujitsu Laboratories, Fujitsu Limited^{RINGGOLD}, Kawasaki City, Kanagawa 211-0053, Japan; orcid.org/0009-0001-9192-5126; Email: gao.hong@fujitsu.com

Authors

Satoshi Imamura – Computing Laboratory, Fujitsu Laboratories, Fujitsu Limited^{RINGGOLD}, Kawasaki City, Kanagawa 211-0053, Japan; orcid.org/0000-0002-8703-3264

Akihiko Kasagi – Computing Laboratory, Fujitsu Laboratories, Fujitsu Limited^{RINGGOLD}, Kawasaki City, Kanagawa 211-0053, Japan

Eiji Yoshida – Computing Laboratory, Fujitsu Laboratories, Fujitsu Limited^{RINGGOLD}, Kawasaki City, Kanagawa 211-0053, Japan

Complete contact information is available at: <https://pubs.acs.org/doi/10.1021/acs.jctc.3c01190>

Notes

The authors declare no competing financial interest.

■ ACKNOWLEDGMENTS

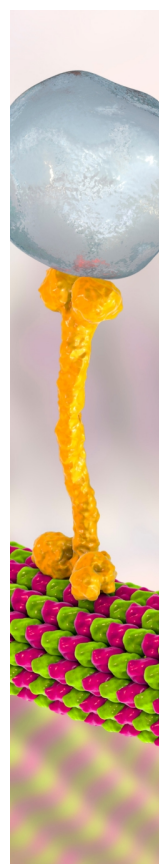
The authors thank the ABCI ground challenge committee for providing an experimental environment and Associate Professor Ishikawa Atsushi of the Tokyo Institute of Technology for valuable advice.

■ REFERENCES

- (1) Karlström, G.; Lindh, R.; Malmqvist, P.-Å.; Roos, B. O.; Ryde, U.; Veryazov, V.; Widmark, P.-O.; Cossi, M.; Schimmelpfennig, B.; Neogrady, P. MOLCAS a program package for computational chemistry. *Comput. Mater. Sci.* **2003**, *28*, 222–239.
- (2) Klamt, A. *COSMO-RS: from Quantum Chemistry to Fluid Phase Thermodynamics and Drug Design*; Elsevier, 2005.
- (3) Zhang, X.-L.; Ma, Y.-T.; Zhai, Y.; Li, H. Full quantum calculation of the rovibrational states and intensities for a symmetric top-linear molecule dimer: Hamiltonian, basis set, and matrix elements. *J. Chem. Phys.* **2019**, *151*, 074301.
- (4) Blunt, N. S.; Camps, J.; Crawford, O.; Izsák, R.; Leontica, S.; Mirani, A.; Moylett, A. E.; Scivier, S. A.; Sünderhau, C.; Schopf, P.; Taylor, J. M.; Holzmman, N. Perspective on the Current State-of-the-Art of Quantum Computing for Drug Discovery Applications. *J. Chem. Theory Comput.* **2022**, *18*, 7001–7023.
- (5) Johnson, E. R.; Becke, A. D. A post-Hartree-Fock model of intermolecular interactions: Inclusion of higher-order corrections. *J. Chem. Phys.* **2006**, *124*, 174104.

- (6) Geerlings, P.; De Proft, F.; Langenaeker, W. Conceptual density functional theory. *Chem. Rev.* **2003**, *103*, 1793–1874.
- (7) Hirao, K. Multireference Møller–Plesset method. *Chem. Phys. Lett.* **1992**, *190*, 374–380.
- (8) Bishop, R. An overview of coupled cluster theory and its applications in physics. *Theor. Chim. Acta* **1991**, *80*, 95–148.
- (9) Christiansen, O.; Koch, H.; Jørgensen, P.; Olsen, J. Excitation energies of H₂O, N₂ and C₂ in full configuration interaction and coupled cluster theory. *Chem. Phys. Lett.* **1996**, *256*, 185–194.
- (10) Eriksen, J. J. The Shape of Full Configuration Interaction to Come. *J. Phys. Chem. Lett.* **2021**, *12*, 418–432.
- (11) Handy, N. C. Multi-root configuration interaction calculations. *Chem. Phys. Lett.* **1980**, *74*, 280–283.
- (12) Knowles, P. J.; Handy, N. C. A new determinant-based full configuration interaction method. *Chem. Phys. Lett.* **1984**, *111*, 315–321.
- (13) Knowles, P. J.; Handy, N. C. Unlimited full configuration interaction calculations. *J. Chem. Phys.* **1989**, *91*, 2396–2398.
- (14) Knowles, P. J.; Handy, N. C. A determinant based full configuration interaction program. *Comput. Phys. Commun.* **1989**, *54*, 75–83.
- (15) Olsen, J.; Jørgensen, P.; Simons, J. Passing the one-billion limit in full configuration-interaction (FCI) calculations. *Chem. Phys. Lett.* **1990**, *169*, 463–472.
- (16) Ansaloni, R.; Bendazzoli, G. L.; Evangelisti, S.; Rossi, E. A parallel Full-CI algorithm. *Comput. Phys. Commun.* **2000**, *128*, 496–515.
- (17) Gan, Z.; Alexeev, Y.; Gordon, M. S.; Kendall, R. A. The parallel implementation of a full configuration interaction program. *J. Chem. Phys.* **2003**, *119*, 47–59.
- (18) Gan, Z.; Harrison, R. Calibrating quantum chemistry: A multi-teraflop, parallel-vector, full-configuration interaction program for the Cray-X1. *Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*, 2005; pp 1–13.
- (19) Rolik, Z.; Szabados, A.; Surjan, P. R. A sparse matrix based full-configuration interaction algorithm. *J. Chem. Phys.* **2008**, *128*, 144101.
- (20) Schriber, J. B.; Evangelista, F. A. Communication: An adaptive configuration interaction approach for strongly correlated electrons with tunable accuracy. *J. Chem. Phys.* **2016**, *144*, 161106.
- (21) Zhang, T.; Evangelista, F. A. A deterministic projector configuration interaction approach for the ground state of quantum many-body systems. *J. Chem. Theory Comput.* **2016**, *12*, 4326–4337.
- (22) Schriber, J. B.; Evangelista, F. A. Adaptive configuration interaction for computing challenging electronic excited states with tunable accuracy. *J. Chem. Theory Comput.* **2017**, *13*, 5354–5366.
- (23) Fales, B. S.; Seritan, S.; Settje, N. F.; Levine, B. G.; Koch, H.; Martínez, T. J. Large-scale electron correlation calculations: Rank-reduced full configuration interaction. *J. Chem. Theory Comput.* **2018**, *14*, 4139–4150.
- (24) Coe, J. P. Machine learning configuration interaction. *J. Chem. Theory Comput.* **2018**, *14*, 5739–5749.
- (25) Wang, Z.; Li, Y.; Lu, J. Coordinate descent full configuration interaction. *J. Chem. Theory Comput.* **2019**, *15*, 3558–3569.
- (26) Tubman, N. M.; Freeman, C. D.; Levine, D. S.; Hait, D.; Head-Gordon, M.; Whaley, K. B. Modern approaches to exact diagonalization and selected configuration interaction with the adaptive sampling CI method. *J. Chem. Theory Comput.* **2020**, *16*, 2139–2159.
- (27) Zhang, N.; Liu, W.; Hoffmann, M. R. Iterative configuration interaction with selection. *J. Chem. Theory Comput.* **2020**, *16*, 2296–2316.
- (28) Abraham, V.; Mayhall, N. J. Selected configuration interaction in a basis of cluster state tensor products. *J. Chem. Theory Comput.* **2020**, *16*, 6098–6113.
- (29) Greene, S. M.; Webber, R. J.; Weare, J.; Berkelbach, T. C. Improved fast randomized iteration approach to full configuration interaction. *J. Chem. Theory Comput.* **2020**, *16*, 5572–5585.
- (30) Chilkuri, V. G.; Neese, F. Comparison of many-particle representations for selected-CI I: A tree based approach. *J. Comput. Chem.* **2021**, *42*, 982–1005.
- (31) Dobroutz, W.; Weser, O.; Bogdanov, N. A.; Alavi, A.; Li Manni, G. Spin-pure stochastic-CASSCF via GUGA-FCIQMC applied to iron–sulfur clusters. *J. Chem. Theory Comput.* **2021**, *17*, 5684–5703.
- (32) Liebermann, N.; Ghanem, K.; Alavi, A. Importance-sampling FCIQMC: Solving weak sign-problem systems. *J. Chem. Phys.* **2022**, *157*, 124111.
- (33) Györfy, W.; Bartlett, R.; Greer, J. Monte Carlo configuration interaction predictions for the electronic spectra of Ne, CH₂, C₂, N₂, and H₂O compared to full configuration interaction calculations. *J. Chem. Phys.* **2008**, *129*, 064103.
- (34) Booth, G.; Thom, A.; Alavi, A. Fermion Monte Carlo without fixed nodes: A game of life, death, and annihilation in Slater determinant space. *J. Chem. Phys.* **2009**, *131*, 054106.
- (35) Cleland, D.; Booth, G. H.; Alavi, A. A study of electron affinities using the initiator approach to full configuration interaction quantum Monte Carlo. *J. Chem. Phys.* **2011**, *134*, 024112.
- (36) Blunt, N. S.; Smart, S. D.; Kersten, J. A. F.; Spencer, J. S.; Booth, G. H.; Alavi, A. Semi-stochastic full configuration interaction quantum Monte Carlo: Developments and application. *J. Chem. Phys.* **2015**, *142*, 184107.
- (37) Blunt, N. S. Communication: An efficient and accurate perturbative correction to initiator full configuration interaction quantum Monte Carlo. *J. Chem. Phys.* **2018**, *148*, 221101.
- (38) Ghanem, K.; Guthrie, K.; Alavi, A. The adaptive shift method in full configuration interaction quantum Monte Carlo: Development and applications. *J. Chem. Phys.* **2020**, *153*, 224115.
- (39) Sun, Q.; Berkelbach, T. C.; Blunt, N. S.; Booth, G. H.; Guo, S.; Li, Z.; Liu, J.; McClain, J.; Sayfutyarova, E. R.; Sharma, S.; Wouters, S.; Chan, G. K.-L. *Python-Based Simulations of Chemistry Framework*; PySCF, 2017, <https://arxiv.org/abs/1701.08223>.
- (40) Sun, Q.; Zhang, X.; Banerjee, S.; Bao, P.; Barbry, M.; Blunt, N. S.; Bogdanov, N. A.; Booth, G. H.; Chen, J.; Cui, Z.-H.; et al. Recent developments in the PySCF program package. *J. Chem. Phys.* **2020**, *153*, 024109.
- (41) mpi4pyscf. An MPI plugin for PySCF. 2020, <https://github.com/pyscf/mpi4pyscf>.
- (42) Peruzzo, A.; McClean, J.; Shadbolt, P.; Yung, M.-H.; Zhou, X.-Q.; Love, P. J.; Aspuru-Guzik, A.; O’Brien, J. L. A variational eigenvalue solver on a photonic quantum processor. *Nat. Commun.* **2014**, *5*, 4213.
- (43) Tilly, J.; Chen, H.; Cao, S.; Picozzi, D.; Setia, K.; Li, Y.; Grant, E.; Wossnig, L.; Rungger, I.; Booth, G. H.; et al. The Variational Quantum Eigensolver: A review of methods and best practices. *Phys. Rep.* **2022**, *986*, 1–128.
- (44) Fedorov, D. A.; Peng, B.; Govind, N.; Alexeev, Y. VQE method: a short survey and recent developments. *Mater. Theory* **2022**, *6*, 2–21.
- (45) Tang, H. L.; Shkolnikov, V.; Barron, G. S.; Grimsley, H. R.; Mayhall, N. J.; Barnes, E.; Economou, S. E. qubit-adapt-vqe: An adaptive algorithm for constructing hardware-efficient ansätze on a quantum processor. *PRX Quantum* **2021**, *2*, 020310.
- (46) Liu, X.; Angone, A.; Shaydulin, R.; Safran, I.; Alexeev, Y.; Cincio, L. Layer VQE: A variational approach for combinatorial optimization on noisy quantum computers. *IEEE Trans. Quant. Eng.* **2022**, *3*, 1–20.
- (47) Chivilikhin, D.; Samarin, A.; Ulyantsev, V.; Iorsh, I.; Oganov, A.; Kyriienko, O. MoG-VQE: Multiobjective genetic variational quantum eigensolver, 2020. <https://arxiv.org/abs/2007.04424>.
- (48) Anselmetti, G.-L. R.; Wierichs, D.; Gogolin, C.; Parrish, R. M. Local, expressive, quantum-number-preserving VQE ansätze for fermionic systems. *New J. Phys.* **2021**, *23*, 113010.
- (49) DiAdamo, S.; Ghibaudi, M.; Cruise, J. Distributed quantum computing and network control for accelerated vqe. *IEEE Trans. Quant. Eng.* **2021**, *2*, 1–21.
- (50) Yang, Y.; Lu, B.-N.; Li, Y. Accelerated quantum Monte Carlo with mitigated error on noisy quantum computer. *PRX Quantum* **2021**, *2*, 040361.
- (51) Huggins, W. J.; O’Gorman, B. A.; Rubin, N. C.; Reichman, D. R.; Babbush, R.; Lee, J. Unbiasing fermionic quantum Monte Carlo with a quantum computer. *Nature* **2022**, *603*, 416–420.

- (52) Zhang, Y.; Huang, Y.; Sun, J.; Lv, D.; Yuan, X. Quantum Computing Quantum Monte Carlo. 2022, <https://arxiv.org/abs/2206.10431>.
- (53) Zhao, T.; Stokes, J.; Veerapaneni, S. Scalable neural quantum states architecture for quantum chemistry. *Mach. Learn. Sci. Technol.* **2023**, *4*, 025034.
- (54) Qiskit contributors Qiskit. An Open-Source Framework for Quantum Computing. 2023, <https://zenodo.org/records/8190968>.
- (55) Davidson, E. R. The iterative calculation of a few of the lowest eigenvalues and corresponding eigenvectors of large real-symmetric matrices. *J. Comput. Phys.* **1975**, *17*, 87–94.
- (56) Fales, B. S.; Levine, B. G. Nanoscale Multireference Quantum Chemistry: Full Configuration Interaction on Graphical Processing Units. *J. Chem. Theory Comput.* **2015**, *11*, 4708–4716.
- (57) Johnson, R. Computational Chemistry Comparison and Benchmark Database. 2018, <http://cccbdb.nist.gov/>.
- (58) ABCI 2.0 User Guide. 2021, <https://docs.abci.ai/en/.2021>.



CAS BIOFINDER DISCOVERY PLATFORM™

BRIDGE BIOLOGY AND CHEMISTRY FOR FASTER ANSWERS

Analyze target relationships,
compound effects, and disease
pathways

Explore the platform

